

AASD 4000

Machine Learning - I

Applied AI Solutions Developer Program



Module 04

Pandas Tutorial

Vejeý Gandýer

Agenda

Installing and Running Pandas

Pandas Objects

Data Indexing and Selection

Missing Data

Combining Datasets

Aggregation and Grouping

Pivot Tables

Strings

Files

Task: Covid Data Analysis

Installing and Running Pandas

Installing Pandas

```
[>>> import pandas
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ModuleNotFoundError: No module named 'pandas'
```

```
Collecting pandas
  Downloading pandas-1.1.5-cp38-cp38-macosx_10_9_x86_64.whl (10.1 MB)
    |████████████████████| 10.1 MB 2.1 MB/s
Collecting numpy>=1.15.4
  Downloading numpy-1.19.4-cp38-cp38-macosx_10_9_x86_64.whl (15.3 MB)
    |████████████████████| 15.3 MB 9.7 MB/s
Collecting python-dateutil>=2.7.3
  Using cached python_dateutil-2.8.1-py2.py3-none-any.whl (227 kB)
Collecting pytz>=2017.2
  Downloading pytz-2020.5-py2.py3-none-any.whl (510 kB)
    |████████████████████| 510 kB 10.4 MB/s
Collecting six>=1.5
  Using cached six-1.15.0-py2.py3-none-any.whl (10 kB)
Installing collected packages: six, pytz, python-dateutil, numpy, pandas
Successfully installed numpy-1.19.4 pandas-1.1.5 python-dateutil-2.8.1 pytz-2020.5 six-1.15.0
```

Conda / Miniconda:
conda search pandas
conda install pandas

Virtual environment:
pip install pandas

Running Pandas

```
import pandas  
pandas.__version__
```

Importing pandas
Checking the version

```
import pandas as pd
```

Aliasing



Getting Help

pd.<TAB>

[In [5]: pd.]

api	BooleanDtype	compat	cut	DatetimeIndex
array()	Categorical	concat	DataFrame	DatetimeTZDtype
arrays	CategoricalDtype	core	date_range	describe_option
bdate_range()	CategoricalIndex	crosstab	DateOffset	errors

Getting Help

pd?

```
Type:      module
String form: <module 'pandas' from '/usr/local/lib/python3.7/site-packages/pandas/__init__.py'>
File:      /usr/local/lib/python3.7/site-packages/pandas/__init__.py
Docstring:
pandas - a powerful data analysis and manipulation library for Python
=====
```

****pandas**** is a Python package providing fast, flexible, and expressive data structures designed to make working with "relational" or "labeled" data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, ****real world**** data analysis in Python. Additionally, it has the broader goal of becoming ****the most powerful and flexible open source data analysis / manipulation tool available in any language****. It is already well on its way toward this goal.

Main Features

Here are just a few of the things that pandas does well:

- Easy handling of missing data in floating point as well as non-floating point data.
- Size mutability: columns can be inserted and deleted from DataFrame and higher dimensional objects
- Automatic and explicit data alignment: objects can be explicitly aligned to a set of labels, or the user can simply ignore the labels and let 'Series', 'DataFrame', etc. automatically align the data for you in computations.
- Powerful, flexible group by functionality to perform split-apply-combine operations on data sets, for both aggregating and transforming data.
- Make it easy to convert ragged, differently-indexed data in other Python and NumPy data structures into DataFrame objects.
- Intelligent label-based slicing, fancy indexing, and subsetting of large data sets.
- Intuitive merging and joining data sets.
- Flexible reshaping and pivoting of data sets.
- Hierarchical labeling of axes (possible to have multiple labels per tick).
- Robust IO tools for loading data from flat files (CSV and delimited), Excel files, databases, and saving/loading data from the ultrafast HDF5 format.
- Time series-specific functionality: date range generation and frequency conversion, moving window statistics, date shifting and lagging.

Pandas Objects

Pandas Objects

Objects: NumPy Structured arrays

Pandas Data Structures

- Series
- DataFrame
- Index

Pandas Objects

Series



Pandas Series

```
data = pd.Series([0.25, 0.5, 0.75, 1.0])  
data
```

Series: One dimensional array of indexed data,
Created from a list or array

```
data.values
```

```
data.index
```

Wraps a sequence of values
and a sequence of indices

```
data[1]
```

```
data[1:3]
```

Can be accessed like an
array



Series as a generalized NumPy array

```
data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                 index=['a', 'b', 'c', 'd'])  
data
```

`data['b']` NumPy array: Implicit index

Series: Explicit index

```
data = pd.Series([0.25, 0.5, 0.75, 1.0],  
                 index=[2, 5, 3, 7])  
data
```

`data[5]` Series Index can be strings

Series Index can be non-continuous integers

Series as a Dictionary

```
population_dict = {'California': 38332521,  
                  'Texas': 26448193,  
                  'New York': 19651127,  
                  'Florida': 19552860,  
                  'Illinois': 12882135}  
population = pd.Series(population_dict)  
population
```

Series: Mappings from a set of key-typed keys to a set of typed values

```
population['California']
```

```
population['California':'Illinois']
```

Constructing Series

```
pd.Series([2, 4, 6])
```

```
pd.Series(5, index=[100, 200, 300])
```

```
pd.Series({2:'a', 1:'b', 3:'c'})
```

```
pd.Series({2:'a', 1:'b', 3:'c'}, index=[3, 2])
```

Constructing Series:

```
pd.Series(data, index=index)
```

data (required) - list or NumPy array

index (optional)

Pandas Objects

DataFrame



Pandas DataFrame as a NumPy array

```
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,  
            'Florida': 170312, 'Illinois': 149995}
```

```
area = pd.Series(area_dict)  
area
```

```
population_dict = {'California': 38332521,  
                  'Texas': 26448193,  
                  'New York': 19651127,  
                  'Florida': 19552860,  
                  'Illinois': 12882135}
```

```
population = pd.Series(population_dict)  
population
```

```
states = pd.DataFrame({'population': population,  
                      'area': area})
```

```
states
```

DataFrame: Two dimensional array with flexible row indices and flexible column names

A Sequence of aligned
(same index) Series
states.index states.columns
objects

Pandas as a Dictionary

```
states = pd.DataFrame({'population': population,  
                        'area': area})
```

```
states
```

```
:      area  population  
California 423967   38332521  
Florida   170312   19552860  
Illinois  149995   12882135  
New York  141297   19651127  
Texas     695662   26448193
```

```
states['area']
```

DataFrame: Maps a column name to a Series of column data

NumPy Array

Data[0] --> first row

DataFrame

Data['Col 0'] --> first column

Constructing DataFrames

1. From a Single Series Object
2. From a list of dicts
3. From a dict of Series objects
4. From a 2D NumPy array
5. From a NumPy Structured array

From a Single Series Object

```
population_dict = {'California': 38332521,  
                  'Texas': 26448193,  
                  'New York': 19651127,  
                  'Florida': 19552860,  
                  'Illinois': 12882135}  
population = pd.Series(population_dict)
```

DataFrame is a collection
of Series Objects

A Single column
dataFrame is constructed
with a Single Series Object

```
pd.DataFrame(population, columns=['population'])
```



From a list of dicts

```
data = [{'a': i, 'b': 2 * i}
        for i in range(3)]
pd.DataFrame(data)
```

DataFrame is a collection of list of dictionaries

```
pd.DataFrame([{'a': 1, 'b': 2}, {'b': 3, 'c': 4}])
```

DataFrame is constructed with **NaN** added for some missing values



From a dictionary of Series Objects

```
area_dict = {'California': 423967, 'Texas': 695662, 'New York': 141297,  
            'Florida': 170312, 'Illinois': 149995}  
  
area = pd.Series(area_dict)  
area
```

DataFrame is a collection
of dictionary of Series
Objects

```
pd.DataFrame({'population': population,  
            'area': area})
```

```
population_dict = {'California': 38332521,  
                  'Texas': 26448193,  
                  'New York': 19651127,  
                  'Florida': 19552860,  
                  'Illinois': 12882135}  
  
population = pd.Series(population_dict)
```

From a 2D NumPy array

```
pd.DataFrame(np.random.rand(3, 2),  
             columns=['foo', 'bar'],  
             index=['a', 'b', 'c'])
```

Given a 2D NumPy array, a DataFrame can be created with specified **columns** and **index**



From a NumPy Structured array

```
A = np.zeros(3, dtype=[('A', 'i8'), ('B', 'f8')])  
A
```

Given a NumPy Structured array, a DataFrame can be created directly from it

```
pd.DataFrame(A)
```

Pandas Objects

Index



Index Object

```
ind = pd.Index([2, 3, 5, 7, 11])  
ind
```

Immutable Array
Ordered Set

Index is created using a
list of integers



Index as Immutable Array

```
Int64Index([2, 3, 5, 7, 11], dtype='int64')
```

```
ind[1]
```

```
ind[:,2]
```

Index object operates like an array

```
print(ind.size, ind.shape, ind.ndim, ind.dtype)
```

Familiarity with NumPy arrays

```
ind[1] = 0
```

Immutable indices

```
TypeError: Index does not support mutable operations
```



Index as Ordered Set

```
indA = pd.Index([1, 3, 5, 7, 9])  
indB = pd.Index([2, 3, 5, 7, 11])
```

```
indA & indB
```

```
indA | indB
```

```
indA ^ indB
```

Index objects are used as joins on datasets

Set operations:

- Union
- Intersection
- Difference
- Symmetric Difference

Data Indexing and Selection

Data Selection

Data Selection

- Series
 - Dictionary
 - 1D array
- DataFrame
 - Dictionary
 - 2D array

Data Selection in Series as a Dictionary

```
import pandas as pd
data = pd.Series([0.25, 0.5, 0.75, 1.0],
                 index=['a', 'b', 'c', 'd'])
data
```

```
data['b']
```

Mapping from a collection of keys to collection of values

```
'a' in data
```

```
data['e'] = 1.25
data
```

Python-like expressions

```
list(data.items())
```

```
data.keys()
```

Mutability possible



Data Selection in Series as a 1D array

```
a    0.25  
b    0.50  
c    0.75  
d    1.00  
e    1.25  
dtype: float64
```

```
data['a':'c']
```

```
data[0:2]
```

```
data[(data > 0.3) & (data < 0.8)]
```

```
data[['a', 'e']]
```

Array-style item selection like

- Slicing
 - Explicit index
 - Implicit index
- Masking
- Fancy Indexing

Data Selection in Series as a 1D array

```
data = pd.Series(['a', 'b', 'c'], index=[1, 3, 5])  
data
```

Explicit and Implicit
indexing confusion

```
data.loc[1]
```

```
data.loc[1:3]
```

loc - Explicit indexing
iloc - Implicit indexing

```
data.iloc[1]
```

```
data.iloc[1:3]
```

Data Selection in DataFrame as a Dictionary

```
area = pd.Series({'California': 423967, 'Texas': 695662,
                  'New York': 141297, 'Florida': 170312,
                  'Illinois': 149995})
pop = pd.Series({'California': 38332521, 'Texas': 26448193,
                 'New York': 19651127, 'Florida': 19552860,
                 'Illinois': 12882135})
data = pd.DataFrame({'area':area, 'pop':pop})
data
```

Mapping from a collection
of keys to collection of
value~

`data.area` `data['area']`

```
data['density'] = data['pop'] / data['area']
data
```

`data.area` **is** `data['area']`

`data.pop` **is** `data['pop']`

Data Selection in DataFrame as a 2D array

	area	pop	density
California	423967	38332521	90.413926
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763
New York	141297	19651127	139.076746
Texas	695662	26448193	38.018740

`data.T`

`data.iloc[:3, :2]`

`data.loc[:'Illinois', :'pop']`

`data.iloc[0, 2] = 90`
`data`

`data.loc[data.density > 100, ['pop', 'density']]`

Data Selection in DataFrame

	area	pop	density
California	423967	38332521	90.413926
Florida	170312	19552860	114.806121
Illinois	149995	12882135	85.883763
New York	141297	19651127	139.076746
Texas	695662	26448193	38.018740

Indexing --> Columns
Slicing --> Rows

```
data['Florida':'Illinois']
```

```
data[1:3]
```

```
data[data.density > 100]
```

Missing Data



Missing Data

Real-world data is not clean and comes with enormous missing values

NaN, None

2 Strategies

- **Mask** (globally indicate missing value)
- **Sentinel value** (choose a value to indicate missing value)

Missing Data

```
vals1 = np.array([1, None, 3, 4])  
vals1
```

```
vals1.sum()
```

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'

```
vals2 = np.array([1, np.nan, 3, 4])  
vals2.dtype
```

```
vals2.sum(), vals2.min(), vals2.max()
```

Missing Data

Typeclass	Conversion when storing NAs	NA sentinel value
floating	No change	np.nan
object	No change	None or np.nan
integer	Cast to float64	np.nan
boolean	Cast to object	None or np.nan

isnull()
notnull()
dropna()
fillna()

isnull()

Generate a Boolean mask indicating missing values

notnull()

Opposite of isnull()

dropna()

Return a filtered version of the data

fillna()

Return a copy of the data with missing values filled or imputed

Detecting null values

```
data = pd.Series([1, np.nan, 'hello', None])
```

```
data.isnull()
```

```
data[data.notnull()]
```

Dropping null values

```
df = pd.DataFrame([[1, np.nan, 2],  
                  [2, 3, 5],  
                  [np.nan, 4, 6]])  
  
df
```

Drop full row or full
column

```
df.dropna()
```

```
df.dropna(axis='columns')
```

Filling null values

```
data = pd.Series([1, np.nan, 2, None, 3], index=list('abcde'))
data
```

```
data.fillna(0)
```

Zero fill

Forward fill

Backward fill

```
data.fillna(method='ffill')    data.fillna(method='bfill')
```

```
df
```

	0	1	2	3
0	1.0	NaN	2	NaN
1	2.0	3.0	5	NaN
2	NaN	4.0	6	NaN

```
df.fillna(method='ffill', axis=1)
```

Combining Datasets

Concat and Append



Concat

```
def make_df(cols, ind):  
    """Quickly make a DataFrame"""  
    data = {c: [str(c) + str(i) for i in ind]  
            for c in cols}  
    return pd.DataFrame(data, ind)  
  
# example DataFrame  
make_df('ABC', range(3))
```

Concatenation of NumPy arrays

```
x = [1, 2, 3]  
y = [4, 5, 6]  
z = [7, 8, 9]  
np.concatenate([x, y, z])
```

```
x = [[1, 2],  
     [3, 4]]  
np.concatenate([x, x], axis=1)
```

Concat

```
pd.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False,  
          keys=None, levels=None, names=None, verify_integrity=False,  
          copy=True)
```

```
ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])  
ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])  
pd.concat([ser1, ser2])
```

```
df1 = make_df('AB', [1, 2])  
df2 = make_df('AB', [3, 4])  
print(df1); print(df2); print(pd.concat([df1, df2]))
```

Duplicate Indices

```
x = make_df('AB', [0, 1])
y = make_df('AB', [2, 3])
```

```
y.index = x.index # make duplicate indices!
print(x); print(y); print(pd.concat([x, y]))
```

x			y			pd.concat([x, y])		
	A	B		A	B		A	B
0	A0	B0	0	A2	B2	0	A0	B0
1	A1	B1	1	A3	B3	1	A1	B1
						0	A2	B2
						1	A3	B3

df1			df2		
	A	B		A	B
1	A1	B1	3	A3	B3
2	A2	B2	4	A4	B4

df1.append(df2)



Combining Datasets

Merge and Join



Merge

`pd.merge()`

Categories of join

- One-to-one
- Many-to-one
- Many-to-many

One-to-one joins

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],  
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})  
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'],  
                    'hire_date': [2004, 2008, 2012, 2014]})  
print(df1); print(df2)
```

df1			df2		
	employee	group		employee	hire_date
0	Bob	Accounting	0	Lisa	2004
1	Jake	Engineering	1	Bob	2008
2	Lisa	Engineering	2	Jake	2012
3	Sue	HR	3	Sue	2014

```
df3 = pd.merge(df1, df2)  
df3
```

Simplest join

Column-wise
concatenation

Many-to-one joins

One of the two key columns has duplicate entries

```
df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],  
                    'supervisor': ['Carly', 'Guido', 'Steve']})  
print(df3); print(df4); print(pd.merge(df3, df4))
```

df3				df4		
	employee	group	hire_date		group	supervisor
0	Bob	Accounting	2008	0	Accounting	Carly
1	Jake	Engineering	2012	1	Engineering	Guido
2	Lisa	Engineering	2004	2	HR	Steve
3	Sue	HR	2014			

```
pd.merge(df3, df4)
```

Many-to-many joins

If both the key columns
has duplicate entries

```
df5 = pd.DataFrame({'group': ['Accounting', 'Accounting',  
                             'Engineering', 'Engineering', 'HR', 'HR'],  
                   'skills': ['math', 'spreadsheets', 'coding', 'linux',  
                              'spreadsheets', 'organization']})  
  
print(df1); print(df5); print(pd.merge(df1, df5))
```

df1			df5		
	employee	group		group	skills
0	Bob	Accounting	0	Accounting	math
1	Jake	Engineering	1	Accounting	spreadsheets
2	Lisa	Engineering	2	Engineering	coding
3	Sue	HR	3	Engineering	linux
			4	HR	spreadsheets
			5	HR	organization

```
pd.merge(df1, df5)
```

on Keyword

df1			df2		
	employee	group		employee	hire_date
0	Bob	Accounting	0	Lisa	2004
1	Jake	Engineering	1	Bob	2008
2	Lisa	Engineering	2	Jake	2012
3	Sue	HR	3	Sue	2014

```
pd.merge(df1, df2, on='employee')
```

This works only if both left and right dataframe has the same column name

left_on & right_on

df1			df3		
	employee	group		name	salary
0	Bob	Accounting	0	Bob	70000
1	Jake	Engineering	1	Jake	80000
2	Lisa	Engineering	2	Lisa	120000
3	Sue	HR	3	Sue	90000

```
pd.merge(df1, df3, left_on="employee", right_on="name")
```

Sometimes, the same column name won't be there in different dataframes.

```
pd.merge(df1, df3, left_on="employee", right_on="name").drop('name', axis=1)
```

inner join

df6		df7			
	name	food		name	drink
0	Peter	fish	0	Mary	wine
1	Paul	beans	1	Joseph	beer
2	Mary	bread			

```
pd.merge(df6, df7, how='inner')
```

intersection

outer join

df6			df7		
	name	food		name	drink
0	Peter	fish	0	Mary	wine
1	Paul	beans	1	Joseph	beer
2	Mary	bread			

```
pd.merge(df6, df7, how='outer')
```

Returns the join over the union of the input columns, fill all the missing values with NaN

left join

df6			df7		
	name	food		name	drink
0	Peter	fish	0	Mary	wine
1	Paul	beans	1	Joseph	beer
2	Mary	bread			

```
pd.merge(df6, df7, how='left')
```

Returns the join over the left entries, fill missing values with NaN

Overlapping Column names

df8			df9		
	name	rank		name	rank
0	Bob	1	0	Bob	3
1	Jake	2	1	Jake	1
2	Lisa	3	2	Lisa	4
3	Sue	4	3	Sue	2

```
pd.merge(df8, df9, on="name")
```

As the column names have conflicts, the join produces two columns with suffixes `_x` and `_y`

Aggregation and Grouping



Aggregation

Aggregation	Description
<code>count()</code>	Total number of items
<code>first(), last()</code>	First and last item
<code>mean(), median()</code>	Mean and median
<code>min(), max()</code>	Minimum and maximum
<code>std(), var()</code>	Standard deviation and variance
<code>mad()</code>	Mean absolute deviation
<code>prod()</code>	Product of all items
<code>sum()</code>	Sum of all items

Instant insight for a dataset

- `sum`
- `median`
- `mean`
- `min`
- `max`
- `describe`
- `groupby`

Planets dataset

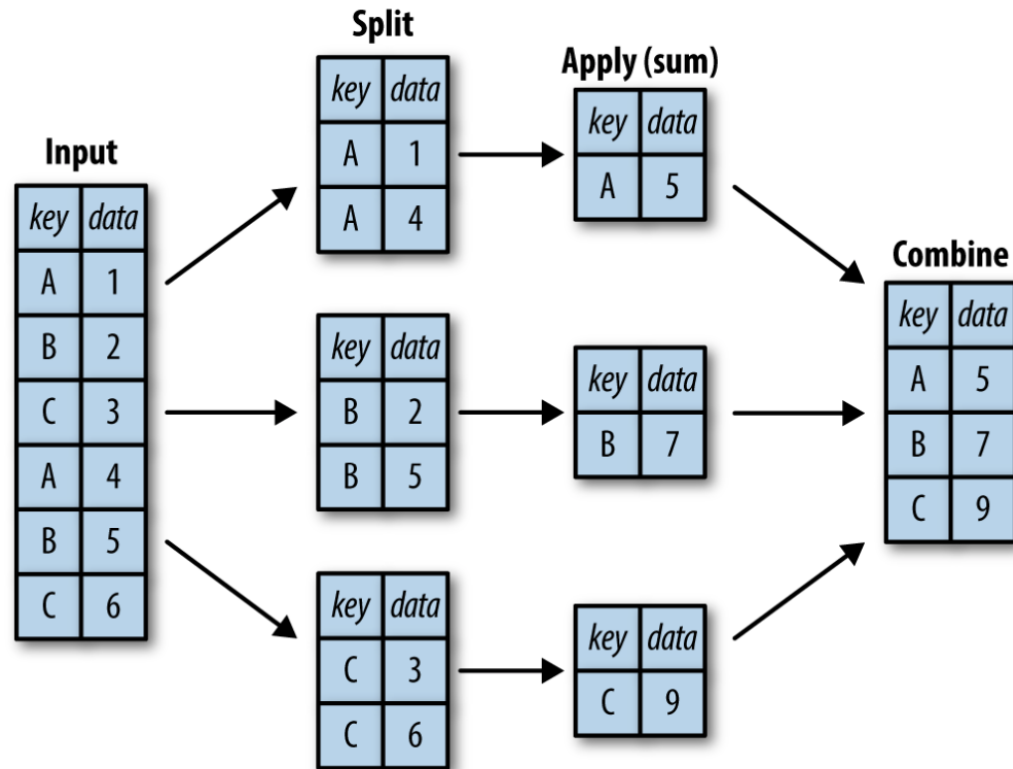
```
import seaborn as sns
planets = sns.load_dataset('planets')
planets.shape
planets.head()
```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

Seaborn (visualization library)

- install through pip
- import library
- load dataset

Groupby



Simple aggregation doesn't give away too much

Need to aggregate conditionally on a label or index : Groupby

Groupby

- Split
- Apply
- Combine

Groupby

```
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],  
                  'data': range(6)}, columns=['key', 'data'])  
df
```

```
df.groupby('key')
```

```
df.groupby('key').sum()
```

Groupby on key "key"

Lazy computation

sum() can be replaced
with any aggregation
method in Pandas or
NumPy

Column Indexing

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

Column Indexing

```
planets.groupby('method')
```

```
planets.groupby('method')['orbital_period']
```

```
planets.groupby('method')['orbital_period'].median()
```

```
method
Astrometry          631.180000
Eclipse Timing Variations  4343.500000
Imaging              27500.000000
Microlensing         3300.000000
Orbital Brightness Modulation    0.342887
Pulsar Timing        66.541900
Pulsation Timing Variations  1170.000000
Radial Velocity       360.200000
Transit              5.714932
Transit Timing Variations   57.011000
Name: orbital_period, dtype: float64
```

Iteration Over Groups

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

Returns each group as a Series or DataFrame

```
for (method, group) in planets.groupby('method'):
    print("{0:30s} shape={1}".format(method, group.shape))
```

Astrometry	shape=(2, 6)
Eclipse Timing Variations	shape=(9, 6)
Imaging	shape=(38, 6)
Microlensing	shape=(23, 6)
Orbital Brightness Modulation	shape=(3, 6)
Pulsar Timing	shape=(5, 6)
Pulsation Timing Variations	shape=(1, 6)
Radial Velocity	shape=(553, 6)
Transit	shape=(397, 6)
Transit Timing Variations	shape=(4, 6)

Dispatch Methods

```
planets.groupby('method')['year'].describe().unstack()
```

	count	mean	std	min	25%	50%	75%	max
method								
Astrometry	2.0	2011.500000	2.121320	2010.0	2010.75	2011.5	2012.25	2013.0
Eclipse Timing Variations	9.0	2010.000000	1.414214	2008.0	2009.00	2010.0	2011.00	2012.0
Imaging	38.0	2009.131579	2.781901	2004.0	2008.00	2009.0	2011.00	2013.0
Microlensing	23.0	2009.782609	2.859697	2004.0	2008.00	2010.0	2012.00	2013.0
Orbital Brightness Modulation	3.0	2011.666667	1.154701	2011.0	2011.00	2011.0	2012.00	2013.0
Pulsar Timing	5.0	1998.400000	8.384510	1992.0	1992.00	1994.0	2003.00	2011.0
Pulsation Timing Variations	1.0	2007.000000	NaN	2007.0	2007.00	2007.0	2007.00	2007.0
Radial Velocity	553.0	2007.518987	4.249052	1989.0	2005.00	2009.0	2011.00	2014.0
Transit	397.0	2011.236776	2.077867	2002.0	2010.00	2012.0	2013.00	2014.0
Transit Timing Variations	4.0	2012.500000	1.290994	2011.0	2011.75	2012.5	2013.25	2014.0

Returns each group as a Series or DataFrame



Aggregate ()

```
rng = np.random.RandomState(0)
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],
                  'data1': range(6),
                  'data2': rng.randint(0, 10, 6)},
                  columns = ['key', 'data1', 'data2'])
df.groupby('key').aggregate(['min', np.median, max])
```

Input: String, list, function

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

```
df.groupby('key').aggregate({'data1': 'min',
                             'data2': 'max'})
```

Filter ()

```
def filter_func(x):
    return x['data2'].std() > 4

print(df); print(df.groupby('key').std());
print(df.groupby('key').filter(filter_func))
```

df				df.groupby('key').std()		
	key	data1	data2	key	data1	data2
0	A	0	5	A	2.12132	1.414214
1	B	1	0	B	2.12132	4.949747
2	C	2	3	C	2.12132	4.242641
3	A	3	3			
4	B	4	7			
5	C	5	9			

Returns a Boolean value specifying whether the group passed the filtering or not

```
df.groupby('key').filter(filter_func)
```

Transform ()

```
df.groupby('key').transform(lambda x: x - x.mean())
```

Returns a transformed version of full data to recombine

Eg: Shifting center with respect to mean



Apply ()

```
def norm_by_data2(x):
    # x is a DataFrame of group values
    x['data1'] /= x['data2'].sum()
    return x
```

```
print(df); print(df.groupby('key').apply(norm_by_data2))
```

Input: DataFrame

Output: Pandas object or Scalar

df				df.groupby('key').apply(norm_by_data2)			
	key	data1	data2		key	data1	data2
0	A	0	5	0	A	0.000000	5
1	B	1	0	1	B	0.142857	0
2	C	2	3	2	C	0.166667	3
3	A	3	3	3	A	0.375000	3
4	B	4	7	4	B	0.571429	7
5	C	5	9	5	C	0.416667	9

Pivot Tables

Pivot Tables

```
import numpy as np
import pandas as pd
import seaborn as sns
titanic = sns.load_dataset('titanic')
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
	man	True	NaN	Southampton	no	False
	woman	False	C	Cherbourg	yes	False
	woman	False	NaN	Southampton	yes	True
	woman	False	C	Southampton	yes	False
	man	True	NaN	Southampton	no	True

Input: column- wise data

Groups the entries into a two-dimensional table that provides a multidimensional summarization of the data

Pivot tables: multidimensional version of GroupBy aggregation

That is, you split- apply-combine, but both the split and the combine happen across not a one- dimensional index, but across a **two-dimensional grid**

```
titanic.pivot_table('survived', index='sex', columns='class')
```



Pivot Tables by Hand

Hand computation

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone
	man	True	NaN	Southampton	no	False
	woman	False	C	Cherbourg	yes	False
	woman	False	NaN	Southampton	yes	True
	woman	False	C	Southampton	yes	False
	man	True	NaN	Southampton	no	True

```
titanic.groupby('sex')[['survived']].mean()
```

```
titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()
```

Pivot Tables

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

Options

- aggfunc
- margins
- fillna
- dropna

```
DataFrame.pivot_table(data, values=None, index=None, columns=None,
                        aggfunc='mean', fill_value=None, margins=False,
                        dropna=True, margins_name='All')
```

```
titanic.pivot_table(index='sex', columns='class',
                    aggfunc={'survived':sum, 'fare':'mean'})
```

```
titanic.pivot_table('survived', index='sex', columns='class', margins=True)
```

class	fare			survived		
	First	Second	Third	First	Second	Third
female	106.125798	21.970121	16.118810	91.0	70.0	72.0
male	67.226127	19.741782	12.661633	45.0	17.0	47.0

class	First	Second	Third	All
sex				
female	0.968085	0.921053	0.500000	0.742038
male	0.368852	0.157407	0.135447	0.188908
All	0.629630	0.472826	0.242363	0.383838

Case Study: US Birth Dataset

Births Dataset

```
births = pd.read_csv('births.csv')
births.head()
```

	year	month	day	gender	births
0	1969	1	1	F	4046
1	1969	1	1	M	4440
2	1969	1	2	F	4454
3	1969	1	2	M	4548
4	1969	1	3	F	4548

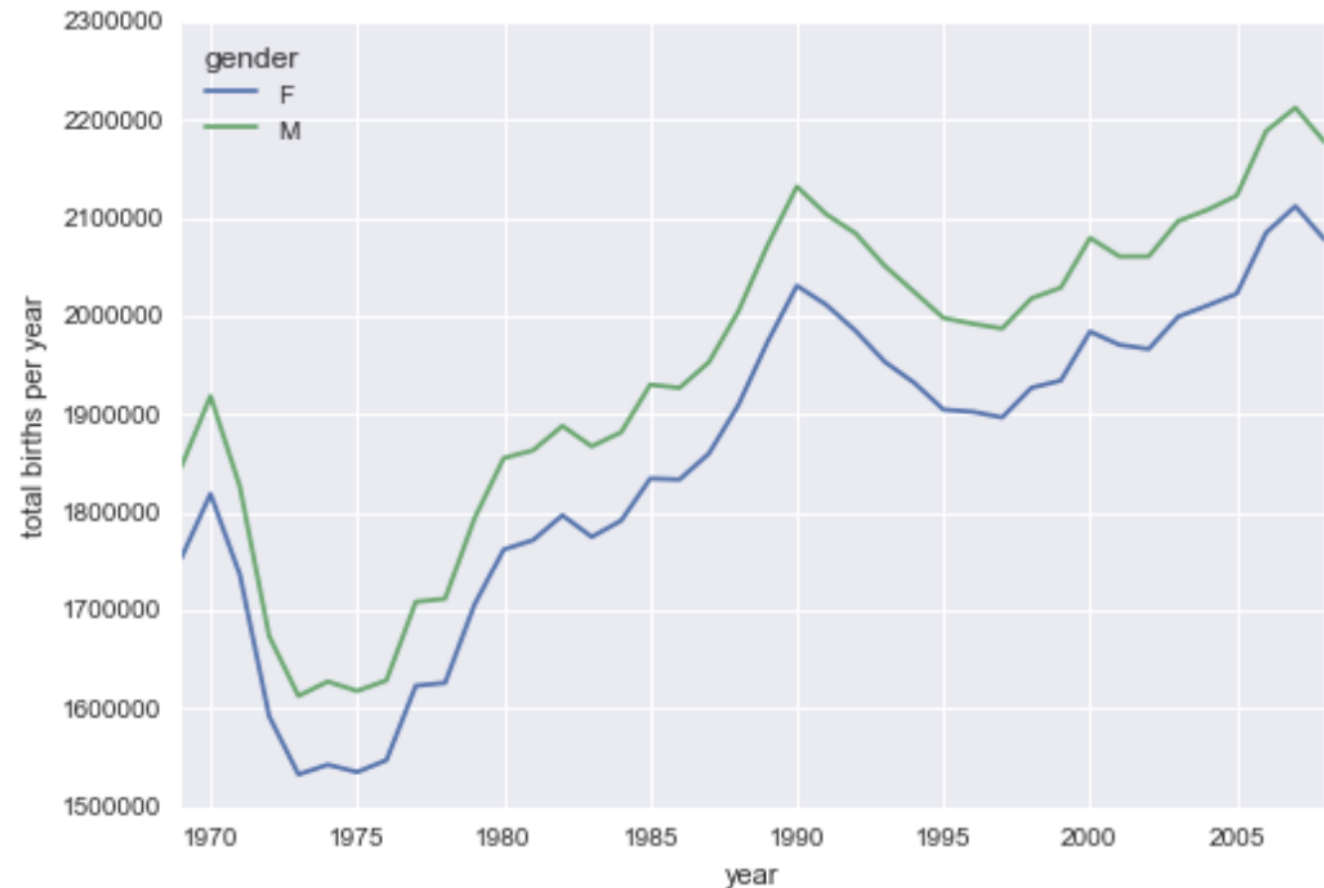
```
births['decade'] = 10 * (births['year'] // 10)
births.pivot_table('births', index='decade', columns='gender', aggfunc='sum')
```

What do you see?

gender	F	M
decade		
1960	1753634	1846572
1970	16263075	17121550
1980	18310351	19243452
1990	19479454	20420553
2000	18229309	19106428

Case Study: US Birth Dataset

```
%matplotlib inline
import matplotlib.pyplot as plt
sns.set() # use Seaborn styles
births.pivot_table('births', index='year', columns='gender', aggfunc='sum').plot()
plt.ylabel('total births per year');
```



Case Study: US Birth Dataset

```
quartiles = np.percentile(births['births'], [25, 50, 75])
mu = quartiles[1]
sig = 0.74 * (quartiles[2] - quartiles[0])

births = births.query('(births > @mu - 5 * @sig) & (births < @mu + 5 * @sig)')

births['day'] = births['day'].astype(int)

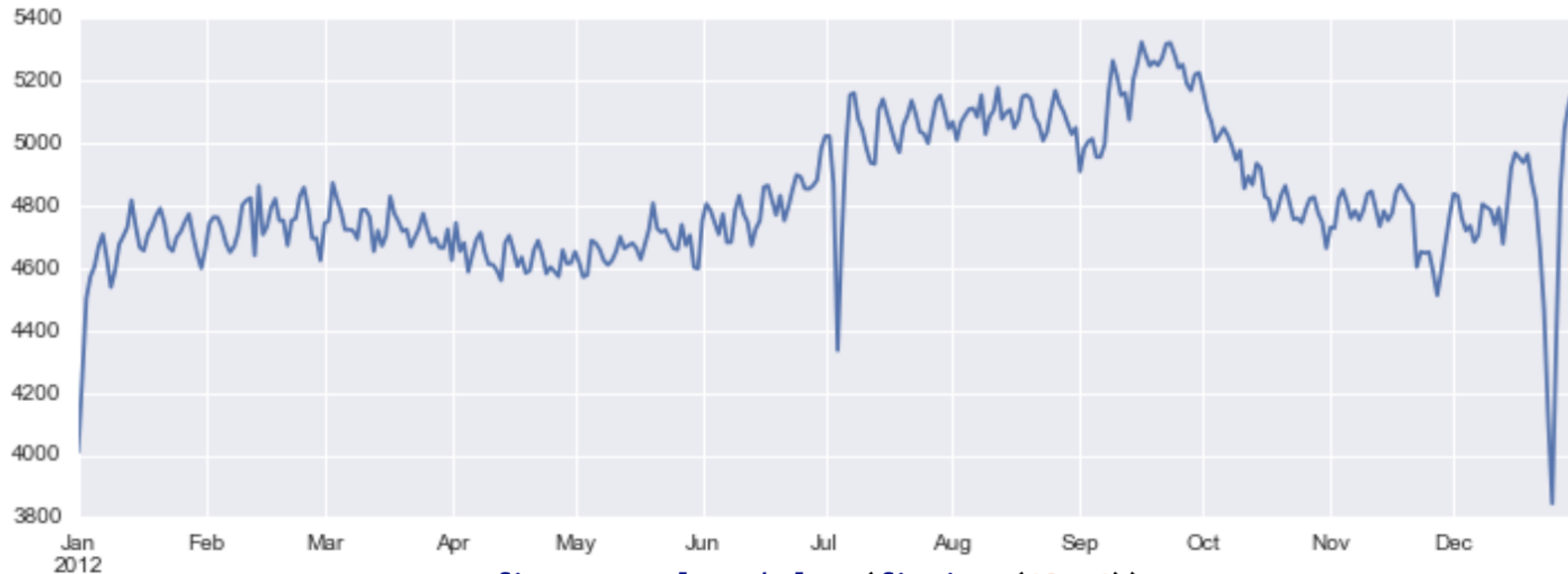
births.index = pd.to_datetime(10000 * births.year +
                              100 * births.month +
                              births.day, format='%Y%m%d')

births['dayofweek'] = births.index.dayofweek
```

Case Study: US Birth Dataset

```
births_by_date = births.pivot_table('births',  
                                     [births.index.month, births.index.day])  
births_by_date.head()
```

```
births_by_date.index = [pd.datetime(2012, month, day)  
                        for (month, day) in births_by_date.index]  
births_by_date.head()
```



```
fig, ax = plt.subplots(figsize=(12, 4))  
births_by_date.plot(ax=ax);
```

Strings



Strings

```
data = ['peter', 'Paul', None, 'MARY', 'gUIDO']
```

```
import pandas as pd  
names = pd.Series(data)  
names
```

```
names.str.capitalize()
```

Vectorization



Strings

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',
                  'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

```
monte.str.lower()
```

```
monte.str.len()
```

String Methods

len()	lower()	translate()	islower()
ljust()	upper()	startswith()	isupper()
rjust()	find()	endswith()	isnumeric()
center()	rfind()	isalnum()	isdecimal()
zfill()	index()	isalpha()	split()
strip()	rindex()	isdigit()	rsplit()
rstrip()	capitalize()	isspace()	partition()
lstrip()	swapcase()	istitle()	rpartition()

```
monte.str.startswith('T')
```

```
monte.str.split()
```

Strings

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',
                  'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

```
monte.str.extract('([A-Za-z]+)')
```

```
monte.str.findall(r'^[^AEIOU].*[^aeiou]$')
```

Regular Expression Methods

Method	Description
<code>match()</code>	Call <code>re.match()</code> on each element, returning a Boolean.
<code>extract()</code>	Call <code>re.match()</code> on each element, returning matched groups as strings.
<code>findall()</code>	Call <code>re.findall()</code> on each element.
<code>replace()</code>	Replace occurrences of pattern with some other string.
<code>contains()</code>	Call <code>re.search()</code> on each element, returning a Boolean.
<code>count()</code>	Count occurrences of pattern.
<code>split()</code>	Equivalent to <code>str.split()</code> , but accepts regexps.
<code>rsplit()</code>	Equivalent to <code>str.rsplit()</code> , but accepts regexps.

Strings

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',  
                  'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

```
monte.str[0:3]
```

```
monte.str.split().str.get(-1)
```

Miscellaneous Methods

Method	Description
get()	Index each element
slice()	Slice each element
slice_replace()	Replace slice in each element with passed value
cat()	Concatenate strings
repeat()	Repeat values
normalize()	Return Unicode form of string
pad()	Add whitespace to left, right, or both sides of strings
wrap()	Split long strings into lines with length less than a given width
join()	Join strings in each element of the Series with passed separator
get_dummies()	Extract dummy variables as a DataFrame

Strings

```
monte = pd.Series(['Graham Chapman', 'John Cleese', 'Terry Gilliam',  
                  'Eric Idle', 'Terry Jones', 'Michael Palin'])
```

```
full_monte = pd.DataFrame({'name': monte,  
                           'info': ['B|C|D', 'B|D', 'A|C', 'B|D', 'B|C',  
                                   'B|C|D']})
```

```
full_monte
```

```
full_monte['info'].str.get_dummies('|')
```

Miscellaneous Methods

Method	Description
get()	Index each element
slice()	Slice each element
slice_replace()	Replace slice in each element with passed value
cat()	Concatenate strings
repeat()	Repeat values
normalize()	Return Unicode form of string
pad()	Add whitespace to left, right, or both sides of strings
wrap()	Split long strings into lines with length less than a given width
join()	Join strings in each element of the Series with passed separator
get_dummies()	Extract dummy variables as a DataFrame

Case Study: Recipe Recommender

```
!curl -O http://openrecipes.s3.amazonaws.com/recipeitems-latest.json.gz  
!gunzip recipeitems-latest.json.gz
```

```
with open('recipeitems-latest.json', 'r') as f:  
    # Extract each line  
    data = (line.strip() for line in f)  
    # Reformat so each line is the element of a list  
    data_json = "[{0}]".format(', '.join(data))  
# read the result as a JSON  
recipes = pd.read_json(data_json)  
  
recipes.shape
```

Parse the recipe data into ingredients list

Given a / a set of ingredients, find all the recipes with it as the ingredient

Case Study: Recipe Recommender

```
recipes.iloc[0]
```

```
recipes.ingredients.str.len().describe()
```

```
recipes.name[np.argmax(recipes.ingredients.str.len())]
```

```
recipes.description.str.contains('[Bb]reakfast').sum()
```

```
recipes.ingredients.str.contains('[Cc]innamon').sum()
```

What is the average size of ingredients?

Which recipe has the longest ingredient?

How many breakfast recipes?

How many recipes contain Cinnamon?

```
recipes.ingredients.str.contains('[Cc]inamon').sum()
```

Case Study: Recipe Recommender

```
spice_list = ['salt', 'pepper', 'oregano', 'sage', 'parsley',
              'rosemary', 'tarragon', 'thyme', 'paprika', 'cumin']

import re
spice_df = pd.DataFrame(
    dict((spice, recipes.ingredients.str.contains(spice, re.IGNORECASE))
         for spice in spice_list))
```

```
spice_df.head()
```

	cumin	oregano	paprika	parsley	pepper	rosemary	sage	salt	tarragon	thyme
0	False	False	False	False	False	False	True	False	False	False
1	False	False	False	False	False	False	False	False	False	False
2	True	False	False	False	True	False	False	True	False	False
3	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False

```
selection = spice_df.query('parsley & paprika & tarragon')
len(selection)
```

Simple Recommender

```
recipes.name[selection.index]
```

2069	All cremat with a Little Gem, dandelion and wa...
74964	Lobster with Thermidor butter
93768	Burton's Southern Fried Chicken with White Gravy
113926	Mijo's Slow Cooker Shredded Beef
137686	Asparagus Soup with Poached Eggs
140530	Fried Oyster Po'boys
158475	Lamb shank tagine with herb tabbouleh
158486	Southern fried chicken in buttermilk
163175	Fried Chicken Sliders with Pickles + Slaw
165243	Bar Tartine Cauliflower Salad
Name: name, dtype: object	

Files



CSV

Reading from a CSV file

```
pd.read_csv("foo.csv")
```

Writing to a CSV file

```
df.to_csv("foo.csv")
```

HDF5

Reading from a HDF5 file

```
pd.read_hdf("foo.h5", "df")
```

Writing to a HDF5 file

```
df.to_hdf("foo.h5", "df")
```

EXCEL

Reading from a Excel file

```
pd.read_excel("foo.xlsx", "Sheet1", index_col=None, na_values=["NA"])
```

Writing to a Excel file

```
df.to_excel("foo.xlsx", sheet_name="Sheet1")
```

References and Further Reading

Pandas user guides

https://pandas.pydata.org/pandas-docs/stable/user_guide/index.html

Python for Data Analysis *Wes McKinney*

Python Data Science Handbook *Jake Vanderplas*

Pandas Tricks

<https://www.dataschool.io/python-pandas-tips-and-tricks/>

Task: Covid-19 Data Analysis

https://github.com/CSSEGISandData/COVID-19/tree/master/csse_covid_19_data/csse_covid_19_daily_reports



Covid Analysis

Data Source: https://github.com/CSSEGISandData/COVID-19/tree/master/csse_covid_19_data/csse_covid_19_daily_reports

Field description

- **Province_State:** China - province name; US/Canada/Australia/ - city name, state/province name; Others - name of the event (e.g., "Diamond Princess" cruise ship); other countries - blank.
- **Country_Region:** country/region name conforming to WHO (will be updated).
- **Last_Update:** MM/DD/YYYY HH:mm (24 hour format, in UTC).
- **Confirmed:** the number of confirmed cases. For Hubei Province: from Feb 13 (GMT +8), we report both clinically diagnosed and lab-confirmed cases. For lab-confirmed cases only (Before Feb 17), please refer to [who covid 19 situation reports](#). For Italy, diagnosis standard might be changed since Feb 27 to "slow the growth of new case numbers." ([Source](#))
- **Deaths:** the number of deaths.
- **Recovered:** the number of recovered cases.

Covid Analysis

Reading the dataset into a Pandas Dataframe

Finding the Number of Confirmed Cases

Finding the Number of Recovered Cases

Finding the Number of Deaths

Finding the Number of Active Cases

Finding the above Country-wise, Province/State-wise

Plotting these insights

